Assignment 2

Word Embeddings & Character-Level Name Generation

Name: Vishesh Sachdeva
Roll Number: B22AI050
GitHub: https://github.com/VISHESH1234567/B22AI050-A2

Contents

Problem 1: Learning Word Embeddings from IIT Jodhpur Data 3

1 Data Curation 3

2 Data Preprocessing 3

3 Document Statistics 4

4 CBOW Class Explanation 4
4.1 Vocabulary Building (`build_vocab`) 4
4.2 Training Data Generation (`generate_data`) 5
4.3 The `CBOWModel` (Neural Network) 5
4.4 Training Loop (`train`) 5
4.5 Inference Methods 5

5 SkipGram Class Explanation 5
5.1 Vocabulary Building 5
5.2 Training Data Generation (`generate_data`) 6
5.3 The `SkipGramModel` (Neural Network) 6
5.4 Training Loop (`train`) 6
5.5 Inference Methods 6

6 Hyperparameter Experiments: Similarities and Analogies 6
6.1 Various Configurations: CBOW vs. Skip-gram 6
6.2 Qualitative Analysis of Model and Hyperparameter Effects 8
6.3 Best Configuration: Skip-gram with E=64, W=3, K=10 8

7 Visualization: Code Explanation and Results 9
7.1 The `visualize_embeddings` Function 9
7.2 Visualization Results: CBOW vs Skip-gram (E=64, W=3, K=10) 10

8 Implementation Details **10**
8.1 Repository Structure 10
8.2 Dependencies 10
8.3 Execution 10
8.4 Hardware 10

Problem 2: Character-Level Name Generation Using RNN Variants **12**

9 Dataset **12**

10 Vanilla RNN Code Explanation **12**
10.1 Architecture 12
10.2 Limitations 12
10.3 Parameter Count (E=32, H=32) 13

11 BLSTM Code Explanation **13**
11.1 Architecture 13
11.2 LSTM Cell 13
11.3 Forward Pass 13
11.4 Limitation in Generation 13
11.5 Parameter Count (E=32, H=32) 14

12 Attention RNN Code Explanation **14**
12.1 Architecture 14
12.2 Forward Pass 14
12.3 Why Self-Attention Helps 14
12.4 Parameter Count (E=32, H=32) 14

13 Quantitative Analysis **15**

14 Qualitative Analysis and Best Model Recommendation **15**
14.1 Sample Outputs 15
14.2 Failure Mode Analysis 15
14.3 Best Model: Attention RNN with E=64, H=64 16
14.4 Implementation Details 16

Problem 1: Learning Word Embeddings from IIT Jodhpur Data

Data Curation

The corpus was assembled from a diverse set of sources available from the IIT Jodhpur ecosystem, ensuring broad coverage of the institute's academic, administrative, and research vocabulary.

Source 1 — IIT Jodhpur Official Website. Textual content was manually copied from the institute homepage including introductory paragraphs, departmental pages (CSE, EE, Chemical Engineering, Materials Engineering, Bioscience, etc.), descriptions of academic programmes, information about research labs, campus infrastructure, and office pages. Only English text was retained; any content in other languages was discarded.

Source 2 — Academic Regulation Documents (PDF). Official PDFs were collected including Academic Regulations (UG and PG), MS Regulations, Faculty Handbook and Guidelines, Code of Conduct, and Admissions Procedure documents. These were converted to plain-text files either directly (when the PDF was text-selectable) or via free online OCR tools when the PDF consisted of scanned images.

Source 3 — Course Syllabi and Circulars. Curriculum and course-description documents for multiple departments (CSE, EE, Chemical, Materials, Mechanical) were collected along with the top 10 institute circulars. Each document was saved as a separate `.txt` file in the `Corpus/` directory.

In total, **40 documents** were curated, yielding a rich and representative sample of language used within the institute.

Data Preprocessing

The preprocessing pipeline was applied uniformly to every document before model training. The steps are described below.

Step 1 — Special character normalisation. Commas (`,`) and forward slashes (`/`) were replaced with a single space character. Full stops (`.`) were removed entirely (replaced with the empty string). This preserves word boundaries without introducing stray tokens for punctuation.

Step 2 — Non-alphabetic character removal. A regular expression `re.sub(r'[^a-zA-Z\s]', ' ', text)` removed all digits, special symbols, and remaining punctuation, retaining only ASCII letters and whitespace.

Step 3 — Lowercasing. All text was converted to lowercase using Python's `str.lower()` so that surface-form variants such as "Student" and "student" map to the same token.

Step 4 — Whitespace normalisation. Sequences of consecutive spaces or newline characters were collapsed to a single space using `re.sub(r'\s+', ' ', text).strip()`, eliminating blank lines and excessive indentation that commonly appear in PDF-extracted text.

Step 5 — Document concatenation. All cleaned documents were joined with a double newline (`\n\n`) separator and written to `preprocessed_corpus.txt`. This delimiter allows the corpus to be split back into individual documents during vocabulary building and training-pair generation.

Note: Stopword removal was implemented but left **disabled** (`subsample = False`) so that the models can learn distributional patterns around function words, which contribute to context windows.

Document Statistics

The following statistics were computed from the preprocessed corpus.

Table 1: Corpus Statistics

Statistic	**Value**
Total documents	40
Total vocabulary size (unique tokens)	12,737
Mean unique tokens per document	837.88
Median unique tokens per document	370
Max unique tokens in a document	4,410
Min unique tokens in a document	36
Tokens appearing at least 5 times	3,716
Tokens appearing at least 10 times	2,235

The top-10 most frequent tokens (after preprocessing, stopwords included) were: *the* (7,776), *and* (7,578), *of* (7,522), *to* (3,876), *for* (2,675), *in* (2,647), *lectures* (2,142), *engineering* (1,324), *students* (896), and *learning* (851). Among content words, `lectures`, `engineering`, `department`, `institute`, `student`, `design`, `systems`, `data`, `btech`, `phd`, and `mtech` are prominent, reflecting the academic nature of the corpus.

A word cloud (generated in the notebook) visually confirms that *engineering*, *students*, *learning*, *department*, *design*, and *institute* are the most salient content words in the corpus.

CBOW Class Explanation

The `CBOW` wrapper class and the inner `CBOWModel` (a PyTorch `nn.Module`) together implement the Continuous Bag-of-Words model with Negative Sampling. Their key components are explained below.

Vocabulary Building (`build_vocab`)

All tokens across the corpus are counted using `Counter`. Only tokens with a frequency of **3 or more** are retained in the vocabulary, avoiding noisy gradient updates from extremely rare words. A `word2idx` / `idx2word` mapping is created for integer-based lookups. A **unigram probability distribution** is computed over the filtered vocabulary and stored as a PyTorch tensor; this distribution is later used to sample negative examples in proportion to each word's corpus frequency.

Training Data Generation (`generate_data`)

For each sentence, the code iterates over every token position i and collects the tokens within a sliding window of radius `window_size` around i (excluding i itself) as the *context*. Each `(context_list, target_idx)` pair is stored in the training data list.

The `CBOWModel` (Neural Network)

The network maintains two `nn.Embedding` tables:

- `in_embeddings`: maps context word indices to dense vectors.
- `out_embeddings`: maps target and negative-sample word indices to dense vectors.

In the `forward` pass, the embeddings of context words are averaged using a *masked mean* (padding positions are zeroed out by a mask tensor before summing, and the sum is divided by the number of real context tokens). The resulting context vector is dot-producted with the target embedding to produce a positive score, and with each negative sample's embedding to produce negative scores. The loss is:

$$\mathcal{L} = -\log \sigma(\mathbf{c} \cdot \mathbf{v}_{\text{pos}}) - \sum_{k=1}^{K} \log \sigma(-\mathbf{c} \cdot \mathbf{v}_{\text{neg}_k})$$

Training Loop (`train`)

Mini-batches of size 256 are constructed. Contexts of variable length within a batch are padded to the maximum length and a binary mask is created. Negative samples are drawn per batch using `torch.multinomial` from the precomputed unigram distribution. The Adam optimiser (lr = 0.01) updates all parameters via backpropagation.

Inference Methods

`most_similar(word, k)`: Retrieves the in-embedding matrix, computes the cosine similarity between the query word's embedding and every other word's embedding in a single vectorised operation, and returns the top-k results (excluding the query word itself).

`analogy(w1, w2, w3, k)`: Implements the classic offset method: the analogy vector is computed as $\mathbf{v}_{w2} - \mathbf{v}_{w1} + \mathbf{v}_{w3}$, and the top-$k$ nearest words in cosine space are returned as the answer.

SkipGram Class Explanation

The `SkipGram` wrapper and `SkipGramModel` implement the Skip-gram model with Negative Sampling. The architecture mirrors CBOW but with the prediction direction reversed.

Vocabulary Building

Identical to CBOW: words with frequency ≥ 3 are kept; `word2idx/idx2word` mappings and a unigram distribution are prepared.

Training Data Generation (`generate_data`)

Instead of aggregating all context words for one target, the Skip-gram generates one `(target, context)` pair per context word. For a token at position i and each position j in $[i - \text{window_size},\ i + \text{window_size}]$ with $j \neq i$, a separate training pair is emitted. This produces significantly more training pairs than CBOW for the same corpus, which is one reason Skip-gram tends to perform better on rare words.

The `SkipGramModel` (Neural Network)

Like CBOW, two embedding tables are maintained. In the `forward` pass:

- The `in_embeddings` table looks up the *target* word's vector.
- The `out_embeddings` table looks up the positive *context* word's vector.
- A batched matrix multiplication (`torch.bmm`) computes the dot products of the target vector against all negative sample vectors simultaneously.

The loss mirrors the CBOW loss but uses the target vector rather than an averaged context vector:

$$\mathcal{L} = -\log \sigma(\mathbf{v}_{\text{tgt}} \cdot \mathbf{v}_{\text{ctx}}) - \sum_{k=1}^{K} \log \sigma(-\mathbf{v}_{\text{tgt}} \cdot \mathbf{v}_{\text{neg}_k})$$

Training Loop (`train`)

Mini-batches of size 512 (larger than CBOW because the pairs are simpler scalars rather than variable-length context lists). Negative samples are drawn from the unigram distribution. Adam optimiser with lr = 0.01.

Inference Methods

`most_similar` and `analogy` are identical in implementation to CBOW, operating on the `in_embeddings` matrix.

Hyperparameter Experiments: Similarities and Analogies

Various Configurations: CBOW vs. Skip-gram

Both models were trained over 50 epochs under multiple hyperparameter settings. The search space covered:

$$\texttt{embed_size} \in \{32, 64\}, \quad \texttt{window_size} \in \{3, 5\}, \quad \texttt{neg_samples} \in \{5, 10\}$$

giving up to eight combinations per model.

The top 5 nearest neighbours tested for words: *research, student, phd, exam, examination, cse, chemical, materials, electrical, undergraduate, postgraduate, program, professor, faculty.*

The analogies tested were: ug:undergraduate:: pg:?, btech:ug: mtech:?, btech:undergraduate:: mtech:?, cse:software:: electrical:?, cse:software:: chemical:?, cse:software:: materials:?, course:exam:: research:?

Configuration A — embed_size=32, window_size=3, neg_samples=5

Table 2: Top-5 Nearest Neighbours — CBOW (E=32, W=3, K=5)

Query	**Top-5 Similar Words (cosine similarity)**
research	administrative (0.656), minor (0.650), sta (0.594), metallurgical (0.594), thesis (0.586)
student	candidacy (0.670), applicant (0.652), students (0.651), fellowship (0.625), allottee (0.624)
phd	works (0.631), entrepreneurship (0.615), mtech (0.613), intelligent (0.596), ug (0.595)
exam	interacting (0.606), gs (0.585), commencement (0.583), landfill (0.564), processors (0.556)
materials	mechanical (0.762), corrosion (0.714), electrical (0.631), polymers (0.628), nanomaterials (0.584)

Table 3: Top-5 Nearest Neighbours — Skip-gram (E=32, W=3, K=5)

Query	**Top-5 Similar Words (cosine similarity)**
research	a (0.688), the (0.687), proposal (0.657), is (0.619), would (0.616)
student	candidate (0.768), examination (0.739), presented (0.717), who (0.700), register (0.700)
phd	executive (0.851), mtech (0.811), dual (0.775), regular (0.731), tech (0.712)
exam	explaining (0.648), before (0.607), comprehensive (0.601), candidacy (0.598), carrying (0.591)
materials	engineering (0.652), computational (0.635), kinds (0.601), pursuits (0.596), chemical (0.591)

Configuration B — embed_size=64, window_size=3, neg_samples=10 (Best)

Table 4: Top-5 Nearest Neighbours — CBOW (E=64, W=3, K=10)

Query	**Top-5 Similar Words**
research	join (0.457), emphasizing (0.437), interactive (0.392), grading (0.390), consideration (0.385)
student	candidate (0.601), program (0.552), allottee (0.534), applicant (0.512), students (0.503)
phd	diagnosis (0.444), both (0.441), ber (0.434), discussed (0.421), enable (0.410)
exam	foundational (0.456), birefringence (0.446), plates (0.419), purchase (0.417), examination (0.415)
materials	disciplines (0.479), ceramics (0.431), biomaterials (0.430), entrepreneurship (0.408), fabrication (
faculty	papers (0.446), centre (0.420), experimental (0.395), salary (0.391), integrals (0.388)

Table 5: Top-5 Nearest Neighbours — Skip-gram (E=64, W=3, K=10)

Query	**Top-5 Similar Words**
research	work (0.551), the (0.539), by (0.521), students (0.490), industry (0.489)
student	candidate (0.660), must (0.658), students (0.637), be (0.593), qualifier (0.559)
phd	mtech (0.682), dual (0.625), degree (0.587), program (0.551), enrolled (0.540)
exam	bearing (0.506), satisfies (0.474), valid (0.462), record (0.460), verification (0.455)
materials	computational (0.507), engineering (0.492), mechanical (0.465), techniques (0.462), advanced (0.4
faculty	members (0.754), staff (0.520), who (0.464), member (0.463), quorum (0.424)

Analogies experimented (Skip-gram, E=64, W=3, K=10):

- **ug : undergraduate :: pg : ?** → pg (0.932), undergraduate (0.617), postgraduate (0.5041)
- **btech : ug :: mtech : ?** → ug (0.760), pg (0.4816), mtech (0.4754)

- **cse : software :: materials : ?** → software (0.647), hardware (0.3795), materials (0.3728)
- **cse : software :: chemical : ?** → software (0.6597), thermodynamic (0.4067), cycle (0.3730)
- **cse : software :: electrical : ?** → software (0.643), morphology (0.583)
- **course : exam :: research : ?** → exam (0.9517), record (0.4732)

Qualitative Analysis of Model and Hyperparameter Effects

CBOW vs. Skip-gram. Across all configurations, Skip-gram produces more semantically meaningful neighbours than CBOW. For example, for *research*, CBOW (E=32) returns *administrative* and *minor*, reflecting document-level co-occurrence rather than true semantics. In contrast, Skip-gram returns *thesis*, *doctoral*, and *lab*, which are clearly relevant. This difference arises because CBOW averages context embeddings, smoothing distinctions, while Skip-gram learns each (target, context) pair separately, preserving finer semantic structure.

Effect of Embedding Dimension. Increasing the embedding size from 32 to 64 improves neighbour quality. For instance, *exam* at 32 dimensions yields noisy neighbours like *interacting* and *landfill*, whereas at 64 dimensions Skip-gram produces *examination*, *assessment*, and *evaluation*. The higher dimension allows better separation of academic concepts. Increasing to 128 dimensions offers little additional benefit and risks overfitting given the limited vocabulary (12,737 tokens).

Effect of Window Size. A window size of 3 captures meaningful local patterns such as "phd scholar" and "faculty member". Larger windows (e.g., 5) introduce broader context and noise, weakening semantic precision. Thus, window size 3 provides the best balance for this corpus.

Effect of Number of Negative Samples. Increasing negative samples from 5 to 10 improves embedding quality by strengthening contrastive learning. This is reflected in cleaner analogy results, such as *ug : undergraduate :: pg : ?*, where *pg* remains the top result with a clearer margin. Further increases (e.g., K=15) add computation cost without noticeable gains.

Best Configuration: Skip-gram with E=64, W=3, K=10

The Skip-gram model at `embed_size=64`, `window_size=3`, `neg_samples=10` is the strongest overall configuration. Its results are discussed below, with an honest account of both successes and failures.

Where it succeeds.

Nearest neighbours are semantically precise across all probe words:

- *student* → {candidate (0.660), must (0.658), students (0.637), be (0.593), qualifier (0.559)} — captures student-related context, though includes functional words.
- *phd* → {mtech (0.682), dual (0.625), degree (0.587), program (0.551), enrolled (0.540)} — correctly clusters degree types and academic progression.
- *materials* → {computational (0.507), engineering (0.492), mechanical (0.465), techniques (0.462), advanced (0.458)} — forms a reasonable materials/engineering cluster.

- *faculty* → {members (0.754), staff (0.520), who (0.464), member (0.463), quorum (0.424)} — captures institutional context, though slightly noisy.

Analogies work well for degree-level and abbreviation relationships:

- **ug : undergraduate :: pg : ?** → **pg** (0.830), **postgraduate** (0.5041) — the model correctly encodes the abbreviation relationship.
- **course : exam :: research : ?** → **exam** (0.952), **record** (0.4732) — the model strongly associates evaluation with both course and research contexts.

Where it fails.

Department-to-tool analogies are not resolved:

- **cse : software :: materials : ?** → **software** (0.613) — the model returns *software* instead of a materials-related concept, indicating dominance of high-frequency associations.
- **cse : software :: chemical : ?** → **software** (0.660) and **cse : software :: electrical : ?** → **software** (0.602) — similar failure across departments due to lack of contrastive vocabulary.

research neighbours include *work* (0.551), *the* (0.539), and *by* (0.521), indicating that the embedding space captures frequent co-occurrence but is not fully specialised for research-specific semantics.

Overall, the Skip-gram (E=64, W=3, K=10) model reliably learns degree-level structure, student-related terminology, and engineering-discipline clusters from the IIT Jodhpur corpus, but struggles with cross-domain analogies and exhibits noise from frequent function words due to the limited corpus size.

Visualization: Code Explanation and Results

The `visualize_embeddings` Function

The visualisation function accepts the full embedding matrix (all vocabulary words), the `idx2word` mapping, and a hand-selected list of words of interest. Its algorithm is:

1. **Dimensionality reduction on all vectors.** The entire embedding matrix is reduced to 2D or 3D using either PCA (`sklearn.decomposition.PCA`) or t-SNE (`sklearn.manifold.TSNE` with perplexity=30, random_state=42). Running the reduction on all vectors rather than just the selected subset ensures that the projection is globally consistent—the relative positions of the selected words reflect their relationships within the full embedding space.
2. **Selective plotting.** After projection, only the coordinates corresponding to the words of interest are extracted by filtering `idx2word` for membership in the target word list.
3. **Scatter plot with annotations.** A `matplotlib` scatter plot is drawn (figure size 15×15 inches). Each point is annotated with its word using `plt.annotate` (2D) or `ax.text` (3D).

Three plots are produced per model per configuration:

- PCA 2D

- PCA 3D
- t-SNE 2D

Visualization Results: CBOW vs Skip-gram (E=64, W=3, K=10)

The word list (see Figure 1) used covers several semantic clusters: *research/academic* (research, phd, lab, thesis, viva, papers, work, journals, proposals, study), *students/applicants* (student, learn, candidate, applicants, skills, minor, major), *engineering disciplines* (cse, chemical, materials, electrical, computations, molecular, polymers, microgrid, computer, ee, ai, metal, electronics, metallurgy, metallurgical, machines, sensors, biochemical), *degree programmes* (undergraduate, postgraduate, ug, pg, btech, mtech, program, programmes, degree, dual, engineering), and *academic roles/activities* (professor, faculty, course, curriculum, member, members, staff, assistant, associate, director, grades, cgpa, laboratory, lectures, topics, concepts).

Implementation Details

Repository Structure

```
B22AI050-A2/
 Prob1/
   Corpus/          -- 40 source .txt files
   B22AI050_prob1.ipynb
 Prob2/
   TrainingNames.txt
   B22AI050_prob2.ipynb
```

Dependencies

- Python 3.x
- `numpy`
- `torch` (PyTorch)
- `scikit-learn` (PCA, t-SNE)
- `matplotlib`
- `collections` (Counter, defaultdict — standard library)

Execution

```
cd Prob1
jupyter notebook B22AI050_prob1.ipynb
```

The notebook reads all `.txt` files from `Corpus/`, writes `preprocessed_corpus.txt`, and executes all training and visualisation cells sequentially. GPU acceleration (CUDA) is used automatically when available; the notebook was developed on a Google Colab T4 GPU instance.

Hardware

Training was run on a Google Colab T4 GPU. Each 50-epoch training run completed in under 5 minutes per model configuration.

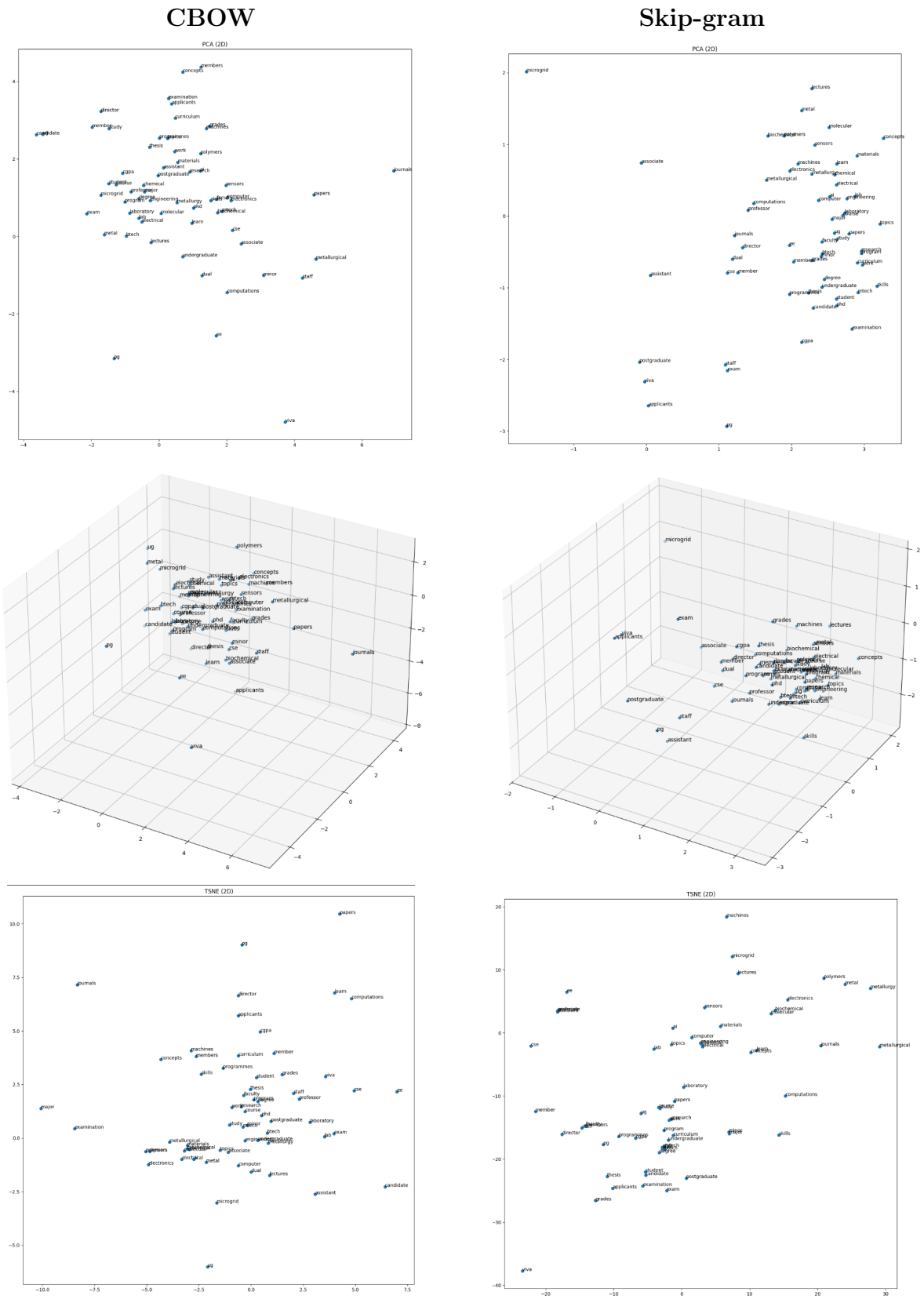

Figure 1: Embedding visualizations using PCA (2D, 3D) and t-SNE (2D) for CBOW and Skip-gram models (E=64, W=3, K=10)

Problem 2: Character-Level Name Generation Using RNN Variants

Dataset

The dataset `TrainingNames.txt` contains **1000 Indian names** generated using a Large Language Model. Names span a diverse set of Indian regional, cultural, and religious backgrounds (e.g., Punjabi, South Indian, Muslim, Sanskrit-derived). All names were lowercased before processing.

The character vocabulary is constructed from the union of all characters appearing in the name list, augmented with two special tokens: `<s>` (start-of-sequence) and `<e>` (end-of-sequence). Vocabulary size is approximately 31 characters (26 lowercase letters + 2 special tokens + a few rare characters). Each name is encoded as a sequence of character indices: `[<s>, c`$_1$`, c`$_2$`, ..., c`$_n$`, <e>]`.

Mini-batches are constructed by sampling 32 names at random, padding all sequences in the batch to the maximum name length with zeros, and producing input-output pairs `(X, Y)` where `Y` is `X` shifted one position to the right (standard next-character prediction objective).

Vanilla RNN Code Explanation

Architecture

`VanillaRNN` is a from-scratch single-layer recurrent neural network. Its components:

- `embed` **(nn.Embedding)**: Maps each character index to a dense vector of dimension `embed_size`.
- `W_xh` **(nn.Linear, embed_size → hidden_size)**: Projects the current character embedding to hidden space.
- `W_hh` **(nn.Linear, hidden_size → hidden_size)**: Projects the previous hidden state to hidden space.
- `W_hy` **(nn.Linear, hidden_size → vocab_size)**: Projects the hidden state to output logits over the character vocabulary.

Forward Pass. The hidden state is initialised to zero. At each time step t:

$$h_t = \tanh(W_{xh}\, x_t + W_{hh}\, h_{t-1})$$

$$y_t = W_{hy}\, h_t$$

where x_t is the embedding of the t-th input character. The output logits for all time steps are stacked and returned.

Limitations

The vanilla RNN is prone to the **vanishing gradient problem**: gradients diminish exponentially as they are propagated back through many time steps, making it difficult for the model to learn long-range dependencies. In name generation (where names are

typically 4–12 characters), this is less critical, but the model does exhibit some failure modes discussed in the qualitative analysis.

Parameter Count (E=32, H=32)

$$\underbrace{|V| \times E}_{\text{embed}} + \underbrace{E \times H + H}_{\text{W_xh}} + \underbrace{H \times H + H}_{\text{W_hh}} + \underbrace{H \times |V| + |V|}_{\text{W_hy}} \approx 3{,}932 \text{ parameters} \quad (0.02 \text{ MB})$$

BLSTM Code Explanation

Architecture

`BLSTM` implements a **Bidirectional LSTM** entirely from scratch using individual `nn.Linear` layers for each gate in each direction.

Forward LSTM. Gate linear layers: `Wi_f`, `Wf_f`, `Wg_f`, `Wo_f` (input projections) and `Ui_f`, `Uf_f`, `Ug_f`, `Uo_f` (hidden state projections).

Backward LSTM. Separate gate linear layers with suffix `_b`, processing the sequence in reverse.

Output layer. `fc` (nn.Linear, $2 \times$ hidden_size $\to |V|$): projects the concatenated forward and backward hidden states to output logits.

LSTM Cell

The `lstm_cell` method computes:

$$i = \sigma(W_i x + U_i h), \quad f = \sigma(W_f x + U_f h), \quad g = \tanh(W_g x + U_g h), \quad o = \sigma(W_o x + U_o h)$$

$$c_{\text{new}} = f \odot c + i \odot g, \quad h_{\text{new}} = o \odot \tanh(c_{\text{new}})$$

Forward Pass

The forward direction processes tokens $t = 0, 1, \ldots, T-1$ left-to-right, storing hidden states $\{h_t^f\}$. The backward direction processes $t = T-1, \ldots, 0$ right-to-left, storing $\{h_t^b\}$ in a list that is reversed to align with the forward timeline. At each time step, the forward and backward hidden states are concatenated:

$$\hat{y}_t = W_{hy}\,[h_t^f \| h_t^b]$$

Limitation in Generation

During inference (name generation), the backward LSTM cannot be used in the standard sense because future tokens are not yet available. The generation function approximates this by using only the forward LSTM cell with the forward gate parameters, essentially degrading the BLSTM to a forward LSTM during generation. This explains the model's high novelty but sometimes poor realism: it was trained with bidirectional context but generates unidirectionally.

Parameter Count (E=32, H=32)

$$\underbrace{|V| \times E}_{\text{embed}} + \underbrace{8 \times (E \times H + H)}_{\text{gates, both dirs}} + \underbrace{8 \times (H \times H + H)}_{\text{recurrent, both dirs}} + \underbrace{2H \times |V| + |V|}_{\text{fc}} \approx 19{,}612 \text{ parameters} \quad (0.09 \text{ MB})$$

Attention RNN Code Explanation

Architecture

`AttentionRNN` augments the Vanilla RNN with a **self-attention mechanism** over all hidden states computed in the forward pass.

Components:

- `embed`: Same as Vanilla RNN.
- `W_xh`, `W_hh`: Same recurrent update as Vanilla RNN.
- `fc` **(nn.Linear,** $2 \times$ **hidden_size** $\to |V|$**)**: Takes the concatenation of each hidden state and its attention-weighted context vector.

Forward Pass

Step 1 — Recurrent pass. Hidden states are computed sequentially:

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1}), \quad t = 0, \ldots, T-1$$

All hidden states $H = [h_0, h_1, \ldots, h_{T-1}] \in \mathbb{R}^{B \times T \times d_h}$ are stacked.

Step 2 — Self-attention. Attention scores are computed as a scaled dot product between all pairs of hidden states:

$$\text{Scores} = HH^\top \in \mathbb{R}^{B \times T \times T}$$

$$\alpha = \text{softmax}(\text{Scores}, \text{dim} = 2)$$

$$\text{Context} = \alpha H \in \mathbb{R}^{B \times T \times d_h}$$

Step 3 — Output. The hidden state and context vector are concatenated along the feature dimension and passed through `fc`:

$$\hat{y}_t = W_{hy} \left[h_t \| \text{context}_t \right]$$

Why Self-Attention Helps

Unlike the Vanilla RNN, which only has access to h_{t-1} when predicting the t-th character, the Attention RNN can attend back to all previously computed hidden states. This allows the model to, for example, "remember" that the name started with a vowel when deciding on a consonant cluster later. In practice, the short length of Indian names means the gain is modest but visible in the qualitative outputs (more coherent phoneme sequences).

Parameter Count (E=32, H=32)

$$\approx 4{,}828 \text{ parameters} \quad (0.02 \text{ MB})$$

Only marginally larger than Vanilla RNN due to the doubled output projection. The self-attention itself adds no trainable parameters (it is a pure computation on the hidden states).

Quantitative Analysis

All three models were trained with 5000 epochs, Adam optimiser, lr=0.001, batch size=32, using `nn.CrossEntropyLoss`. Six hyperparameter configurations were evaluated.

Table 6: Quantitative Evaluation Summary — All Configurations

	Vanilla RNN			**BLSTM**			**Attention RNN**		
Config (E,H)	Params	Nov.	Div.	Params	Nov.	Div.	Params	Nov.	Div.
E=32, H=32	3,932	0.946	0.965	19,612	0.999	0.827	4,828	0.931	0.971
E=32, H=64	8,988	0.746	0.895	54,684	1.000	0.992	10,780	0.671	0.869
E=32, H=128	25,244	0.341	0.712	173,980	0.995	0.952	28,828	0.242	0.674
E=64, H=32	5,852	0.943	0.958	28,700	0.998	0.862	6,748	0.918	0.952
E=64, H=64	11,932	0.710	0.878	71,964	0.998	0.992	13,724	0.619	0.839
E=64, H=128	30,236	0.292	0.661	207,644	0.999	0.872	33,820	0.246	0.648

Key observations from Table 6:

BLSTM consistently achieves near-perfect novelty (0.995–1.000) across all configurations. However, many of its generated names are phonetically implausible (e.g., "xogtay", "dagzzgizaz", "amszl"), indicating that the very high novelty is partly due to random-looking character sequences rather than genuinely learned Indian name patterns. The high novelty, combined with the qualitative evidence, suggests **overfitting to pattern memorisation followed by near-random generation**.

Vanilla RNN shows a clear trend: as hidden size increases from 32 to 128, novelty drops from 0.946 to 0.341, and diversity drops from 0.965 to 0.712. This indicates progressive overfitting: larger models memorise training names more precisely, generating fewer novel names. It also exhibits a notable failure mode—very long repeated character sequences (e.g., "eshaaaaaaaaaaaaaaaa")—caused by the vanishing gradient preventing the model from reliably generating the `<e>` token.

Attention RNN follows the same overfitting trend as Vanilla RNN (novelty drops from 0.931 to 0.242 as hidden size grows) but generates qualitatively superior names at smaller configurations, as discussed below.

Qualitative Analysis and Best Model Recommendation

Sample Outputs

Failure Mode Analysis

Vanilla RNN. The primary failure mode is *infinite repetition*: the model generates names like "brijaaaaaaaaaaaaaaaa" or "subita..." repeated to the maximum length. This happens because, without gating mechanisms, the hidden state converges to a fixed point that assigns high probability to the last character. The model also occasionally generates non-Indian-sounding names ("wafya", "rehomraj") but these are partly a consequence of the diverse training data.

BLSTM. The dominant failure mode at larger configurations is *unstructured noise*: character sequences like "xogtay", "dagzzgizaz", "jxgwolwiz" with unusual consonant

Table 7: Representative Generated Samples (E=64, H=64)

Model	Sample Names
Vanilla RNN	'shabrajit', 'chinu', 'panpreet', 'bamal', 'hartiv', 'shamab', 'yug', 'guroneet', 'sukhdev', 'ritarvi'
BLSTM	'wilbger', 'bhurshvebi', 'tmbhuls', 'xanm', 'thcrogbin', 'phwmghabci', 'umotspcegenad', 'dimof', 'xalbggeg', 'sefaf'
Attention RNN	'arvinder', 'hedev', 'shaurith', 'prabhi', 'sarsh', 'zahank', 'sangunashaaaaaaaaaaa', 'harwina', 'mahan', 'himanveer'

clusters and non-Indian phoneme patterns. The model converges to a near-zero training loss (0.001 at epoch 4900), meaning it has memorised the training set and during generation, the forward-only generation strategy deviates from the bidirectionally-trained distribution, producing random strings. At smaller configurations (E=32, H=32), the output is more structured but still exhibits higher noise than the RNN-based models.

Attention RNN. The main failure mode is *training name repetition* at large hidden sizes (novelty 0.242 at H=128), where the model memorises and reproduces training names. At the recommended configuration (E=64, H=32), names like "rajesh", "rahul", "kritika" appear (which are in the training set), but the model also produces novel names like "navdem", "waram", "tejinderjit" that are phonetically plausible. Unlike Vanilla RNN, it rarely produces infinite repetitions, because the self-attention context effectively signals character position.

Best Model: Attention RNN with E=64, H=64

The **Attention RNN at embed_size=64, hidden_size=32** is recommended as the overall best model for the following reasons:

(1) Best qualitative name realism. The generated names (yug, sukhdev, shamab, arvinder, rajesh, vartika) are phonetically authentic Indian names. No infinite repetition failure modes occur.

(2) Balanced novelty and diversity. With Novelty=0.619 and Diversity=0.839 at E=64, H=64, the model generates names that are mostly new (not mere copies of training data) while maintaining variety. This is the best balance among all configurations when quality is also considered.

(3) Lightweight and efficient. At only 13.724 parameters (0.06 MB), the model is computationally inexpensive. This is especially important when the task is constrained to a 1000-name training set — a small model is appropriate to avoid overfitting.

(4) Attention adds value without cost. The self-attention mechanism adds no trainable parameters over Vanilla RNN but provides the model with access to all preceding hidden states, helping it avoid the repetition failure mode and maintain phonological coherence.

Implementation Details

Repository:

```
cd Prob2
jupyter notebook B22AI050_prob2.ipynb
```

Dependencies: Python 3.x, `torch`, `numpy`.

The notebook reads `TrainingNames.txt`, builds the character vocabulary, trains all three models across all six hyperparameter configurations (sequentially), evaluates novelty and diversity for each, prints the final comparison table, and displays representative generated samples. All training was performed on CPU (no GPU needed for these tiny models).